FOTOMAC — como o sistema foi construído

Guia de estudo pessoal para a apresentação · Thiago Nunes · outubro de 2026

Este guia explica o FOTOMAC por dentro: o que já foi feito, o que falta, como as peças se encaixam e por que cada decisão foi tomada. O objetivo é que você consiga explicar qualquer parte do sistema com as suas próprias palavras. Cada capítulo começa pela ideia e só depois mostra o código.

Sumário

1. O FOTOMAC em uma página
2. O que fizemos, e em que ordem
3. O que está pronto e o que falta
4. A arquitetura: as cinco peças
5. Como as peças se ligam
6. O reconhecimento facial por dentro
7. Os caminhos de ponta a ponta
8. O banco de dados
9. O trabalhador em segundo plano
10. Privacidade desde o desenho (LGPD)
11. Como provamos que funciona
12. Onde está cada coisa no código
13. Limitações que você deve conhecer
14. Perguntas que a banca pode fazer
15. Glossário

---

1. O FOTOMAC em uma página

O problema. O CESMAC fotografa formaturas, congressos e jogos. Depois do evento, a pessoa que quer as próprias fotos teria de folhear centenas de imagens de desconhecidos, o que é lento e expõe todo mundo que aparece nelas.

A solução. A pessoa abre a página do evento e faz uma **captura guiada** pela câmera: segue, com a cabeça, quatro movimentos numa ordem sorteada pelo servidor (a **prova de vida**). O sistema mostra **apenas as fotos em que ela aparece**. Ninguém vê a galeria inteira, e não existe busca por foto enviada da galeria.

Quem usa:

Perfil	O que faz	Precisa de conta?
Visitante	Encontra as próprias fotos pelo rosto, baixa e pede remoção	Não
Fotógrafo	Cria eventos, envia, confere e publica as fotos	Sim, por convite
Marketing	O mesmo que o fotógrafo e, além disso, vê as fotos publicadas de qualquer evento para a divulgação	Sim, por convite
Fotógrafo contratado	Envia fotos a **um** evento, por **24 h**; elas só são publicadas depois da aprovação do administrador	Conta temporária
Administrador	Gerencia contas, temas e parâmetros do reconhecimento; aprova as fotos do contratado	Sim

A frase que resume o projeto: o rosto é um dado pessoal **sensível**, de natureza biométrica (LGPD, Art. 5º, II), então o sistema inteiro foi desenhado para usar o mínimo possível desse dado, pelo menor tempo possível, e só depois que a pessoa declara estar ciente.

---

2. O que fizemos, e em que ordem



Figura 1 — Linha do tempo do desenvolvimento.

O trabalho começou **antes do sistema**, no **fotomac-lab**: uma bancada de testes em que comparamos 5 tecnologias de reconhecimento facial com fotos reais de formatura. O InsightFace venceu e o sistema nasceu com ele. Depois da 1ª banca, o CESMAC trocou para o **YuNet + SFace** (2º colocado), porque a licença dele permite uso institucional sem custo. A troca levou um dia porque o sistema foi construído com adaptador (detalhes no capítulo 6).

Depois, o sistema foi construído em **etapas**. Cada uma foi entregue funcionando e testada antes da próxima começar:

Etapa	O que entregou
1 — A base	Banco de dados, adaptador do reconhecimento facial, ligação com o SeaweedFS
2a — Login	Contas por convite, primeiro acesso, recuperação de senha, bloqueio após 5 erros
2b — Fotógrafo	Eventos, envio em lote, conferência, publicação, fila de indexação facial, ZIP
3 — Visitante	Página pública, "Estou ciente", busca pelo rosto, download, remoção
Correções	Pontos que você reprovou nos testes: o resultado some ao sair; links presos à sessão
4a — Contas	O administrador cria, edita, suspende e desbloqueia contas
4b — Configuração	Temas, parâmetros do reconhecimento, reindexação, expiração automática
Pós 1ª banca	YuNet + SFace no lugar do InsightFace, troca de modelo pelo painel, teste de precisão
Requisitos 2.0	Documento revisado com a banca, o cliente, a coordenação e o marketing
Produção v2.0	Captura guiada com prova de vida; perfis Marketing e fotógrafo contratado; excluir evento com prazo para desfazer; andamento do evento e sino de avisos no painel

O diário completo, com o porquê de cada decisão, está em `HISTORICO.md`.

---

3. O que está pronto e o que falta

O documento de requisitos **v2.0** (seção 9) divide o sistema em **13 fases**. Comparando com o que existe hoje:



Figura 2 — Situação de cada item das fases de entrega.

O que falta, em detalhe:

Item	Requisito	O que já existe	O que falta
Pessoas institucionais	RF020, RF021	Tudo pronto (21/10): cadastro, assinatura na tela, PDF, busca no acervo inteiro sem teto, marcação, confirmação, revogação	O envio real do e-mail (hoje o link sai em modo de demonstração)
Registros e uso atípico	RF024	O registro inalterável (`audit_log`) grava tudo	A tela de consulta, o alerta de uso atípico e a tela de remoções do administrador
Calibragem do limiar	RNF18	O teste de precisão dentro do sistema	Medir com mais pessoas e rever a meta de revocação (80%)
Vídeo	RF034, RF035	—	Prova de conceito; depende de ampliar o armazenamento
Crianças e adolescentes	RF032, RF033	— (tudo foi retirado em 19/10: código, banco, telas e avisos)	**Aguardando a resposta oficial** do orientador e da coordenação ao documento de dúvidas
Implantação	—	Tudo configurável pelo `.env`	E-mail real (SMTP), HTTPS, retirar o link de demonstração

Fora do site: o aplicativo móvel vem depois que o site estiver completo e aprovado.

> **Para a banca:** as fases 1 a 8 estão prontas e testadas. A evolução até a próxima banca é o que está nesta tabela, e a calibragem do limiar é a parte mais "científica" dela.

---

4. A arquitetura: as cinco peças

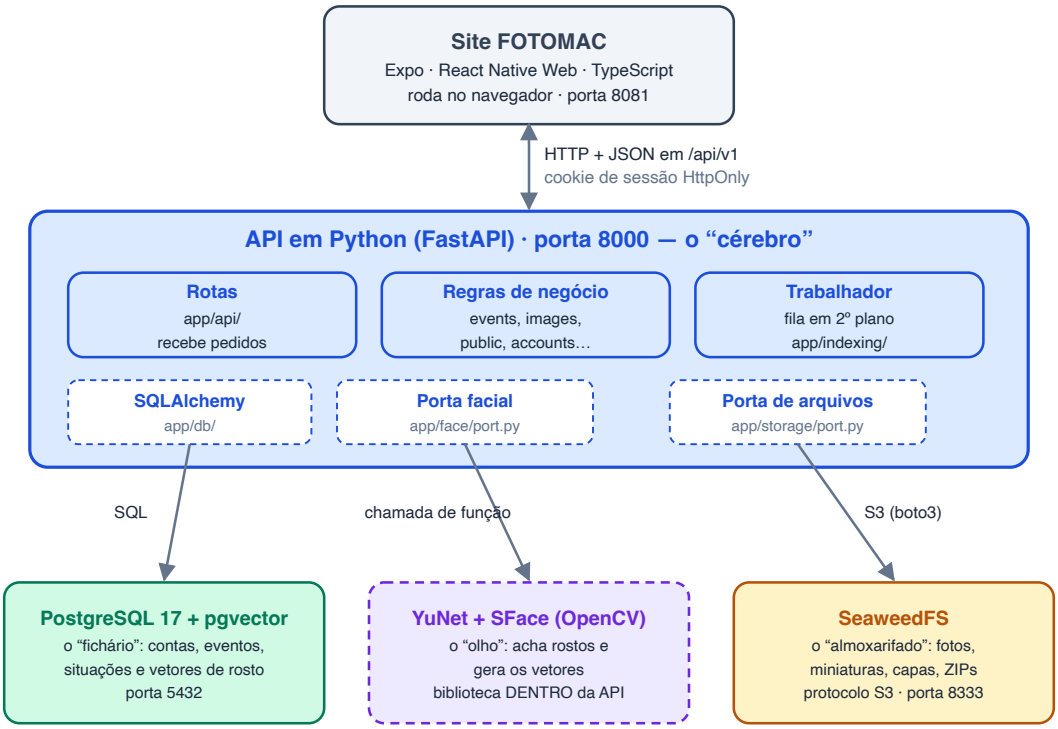


Figura 3 — As cinco peças do sistema e como conversam.

Pense no sistema como um **estúdio fotográfico**:

Peça	Papel no estúdio	O que é de verdade
Site (Expo)	A **vitrine/balcão**: onde as pessoas são atendidas	Telas que rodam no navegador
API Python	O **gerente**: recebe todo pedido, confere quem pediu e decide	Um programa servidor que responde em JSON
PostgreSQL	O **fichário**: as fichas de cada evento, foto e conta	Banco de dados relacional
SeaweedFS	O **almoxarifado**: guarda as caixas (arquivos de foto)	Armazenamento de objetos, protocolo S3
YuNet + SFace	O **olho**: reconhece rostos	Modelos de IA executados pelo OpenCV, dentro da API

Três regras de ouro da arquitetura:

1. **O site nunca fala com o banco nem com o SeaweedFS.** Ele só fala com a API. É a API que confere a permissão antes de qualquer coisa, assim como o balconista não entra no almoxarifado: ele pede ao gerente.
2. **O almoxarifado não sabe o que guarda.** O SeaweedFS só conhece "caixa com tal etiqueta". Quem sabe que a caixa `events/…/abc.jpg` é a foto 37 da formatura é o fichário (PostgreSQL).
3. **O reconhecimento facial não é um servidor.** O YuNet e o SFace são modelos carregados **dentro** do processo da API, pelo OpenCV, e a API chama uma função deles como chamaria qualquer outra função Python. Por isso aparecem tracejados na figura.

As portas na sua máquina:

Serviço	Endereço	Quem acessa
Site	`localhost:8081`	Você, pelo navegador
API	`localhost:8000`	O site
PostgreSQL	`127.0.0.1:5432`	Só a API
SeaweedFS	`127.0.0.1:8333`	Só a API

---

5. Como as peças se ligam

Esta é a pergunta mais importante: *como o Python fala com o SeaweedFS, com o Expo e com o modelo?* Cada ligação usa um **protocolo**, que é um idioma combinado entre dois programas.

5.1 Site (Expo) ↔ API (Python): HTTP e JSON

O site e a API são **dois programas separados**, que podem até estar em computadores diferentes. Eles conversam pela internet, em **HTTP**, trocando mensagens em **JSON** (texto estruturado).

Todo pedido do site passa por **uma única função**, em `frontend/src/api/client.ts`:

```
// A ÚNICA porta das telas para o servidor
const response = await fetch(`${API_URL}/api/v1${path}`, {
  credentials: 'include',     // envia o cookie da sessão
  ...
});
```

Do lado do Python, o **FastAPI** recebe o pedido e escolhe a função pelo endereço:

```
@router.post("/events/{event_id}/search")   # backend/app/api/public.py
def search(...):
    ...
```

- **`/api/v1`:** o "v1" é a versão do contrato. Quando o aplicativo móvel chegar, ele vai usar **exatamente a mesma API**, sem duplicar regra nenhuma.
- **Cookie HttpOnly:** depois do login, a API entrega um cookie que o JavaScript **não consegue ler**. Assim, mesmo que um script malicioso rode na página, ele não rouba a sessão.

- **CORS:** a API só aceita pedidos com cookie vindos do endereço do site (`localhost:8081`). Outro site não consegue usar a sessão de quem está logado.
- **Origem conferida:** além disso, o servidor recusa qualquer ação que altera dados (POST, PUT, DELETE) vinda de outra origem, inclusive de outra página `*.cesmac.edu.br`, que o navegador trataria como "o mesmo site" (proteção contra **CSRF**).

5.2 API ↔ PostgreSQL: SQL, pelo SQLAlchemy

O Python não escreve SQL "na mão" na maior parte do código. Ele usa o **SQLAlchemy**, que traduz classes Python em tabelas:

```
class Image(Base):            # backend/app/db/models.py
    __tablename__ = "images"
    ...
```

A estrutura do banco evolui por **migrações** (`backend/migrations/`, ferramenta **Alembic**). Cada migração é um passo numerado (0001 a 0024), e qualquer computador chega ao mesmo banco rodando `alembic upgrade head`. O **pgvector** é uma extensão do PostgreSQL que cria o tipo `vector` e sabe comparar vetores rapidamente (capítulo 6).

5.3 API ↔ SeaweedFS: o protocolo S3, pelo boto3

S3 é o idioma criado pela Amazon para guardar arquivos ("objetos") em "baldes" (*buckets*). Ele virou padrão de mercado, e o SeaweedFS **fala S3**. Por isso usamos o **boto3**, a biblioteca oficial de S3 para Python. Ela não sabe que do outro lado está o SeaweedFS, e nem precisa saber.

```
# backend/app/storage/s3_storage.py
self._client = boto3.client("s3", endpoint_url=endpoint,
                            aws_access_key_id=access_key,
                            aws_secret_access_key=secret_key)
```

Os endereços e as senhas ficam no arquivo `backend/.env`, que **nunca** vai para o GitHub:

```
FOTOMAC_S3_ENDPOINT=http://127.0.0.1:8333
FOTOMAC_S3_BUCKET=fotomac-acervo
```

Como as fotos ficam organizadas no almoxarifado. Cada foto enviada vira três arquivos:

```
events/<id do evento>/<id da foto>.jpg          ← original (com crédito, sem GPS)
events/<id do evento>/<id da foto>_thumb.jpg    ← miniatura de 480 px
events/<id do evento>/<id da foto>_preview.jpg  ← prévia de 1600 px
```

Os ids são **aleatórios** (UUID), e não 1, 2, 3, para que ninguém consiga adivinhar o endereço da próxima foto.

Por que não guardar as fotos no próprio PostgreSQL? Bancos relacionais são ótimos para fichas pequenas, mas lentos e caros para milhares de arquivos de 10 MB. Além disso, o SeaweedFS **já é o que o servidor do CESMAC usa**.

5.4 API ↔ reconhecimento facial: o adaptador e o builder

Aqui está o pedido do seu orientador: **o sistema não pode depender da biblioteca de IA**. A solução é o padrão **portas e adaptadores**:

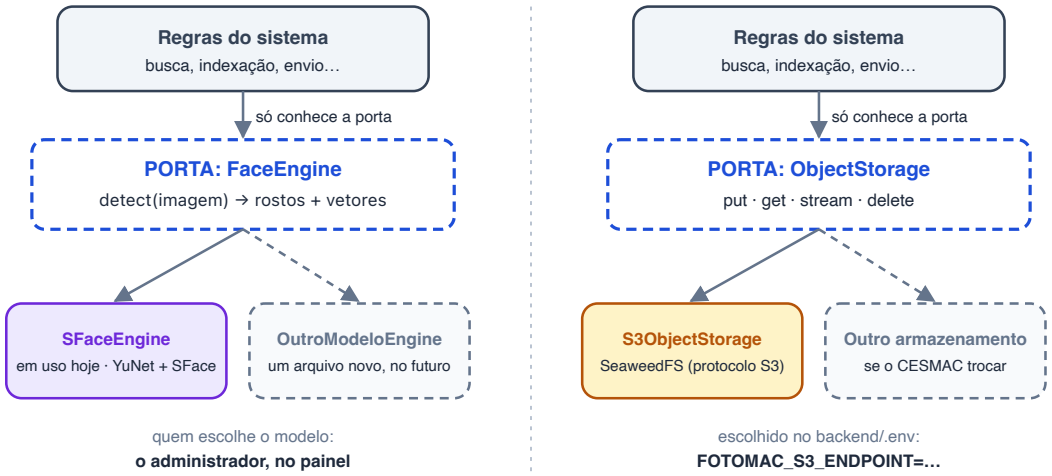


Figura 4 — Portas e adaptadores: o mesmo desenho para o rosto e para os arquivos.

Pense numa **tomada**:

- A **porta** é a tomada da parede. Ela define o formato: "me dê uma imagem e eu devolvo os rostos com os vetores" (`backend/app/face/port.py`). Esse arquivo **não importa nenhuma biblioteca de IA**.
- O **adaptador** é o plugue do aparelho. Hoje há um, `sface_engine.py`, o **único arquivo do sistema inteiro** que carrega o YuNet e o SFace.
- O **builder** (`factory.py`) é o quadro de tomadas: tem o **catálogo** dos modelos que o sistema conhece, cada um com o seu limiar calibrado, e monta o modelo que está em uso. Quem escolhe o modelo é o **administrador, no painel** (Reconhecimento facial); a escolha fica gravada no banco.

Para oferecer outra IA: escreve-se um arquivo novo de adaptador e acrescenta-se uma linha no catálogo do builder. Busca, indexação e telas não mudam. A troca, no painel, segue uma ordem:

1. o modelo novo é **carregado e conferido antes**. Se não carregar, nada muda;
2. a troca fica registrada com o nome do administrador;
3. o limiar volta ao valor **calibrado do modelo novo**, porque cada modelo tem a sua própria escala de semelhança;
4. o acervo publicado entra na fila de **reindexação**. Como vetores de modelos diferentes **não se comparam**, cada vetor guarda a versão do modelo que o gerou, e a foto só volta a aparecer na busca depois de reindexada. Os vetores antigos são apagados assim que os novos existem.

O sistema nunca troca de modelo sozinho. Se o modelo em uso não carregar (arquivo faltando ou adulterado), a busca pausa com um aviso, a fila espera e o painel mostra o motivo. Trocar automaticamente para outro modelo compararia vetores incompatíveis, e um modelo descalibrado mostraria fotos de terceiros.

O **mesmo desenho** vale para os arquivos: se o CESMAC um dia trocar o SeaweedFS, só o adaptador de armazenamento muda.

6. O reconhecimento facial por dentro

6.1 Qual modelo usamos

YuNet + SFace, dois modelos do repositório oficial do OpenCV (*opencv_zoo*), executados pelo **OpenCV 5** na **CPU**, sem placa de vídeo:

Modelo	Arquivo	Função	Licença
YuNet (detector)	`face_detection_yunet_2023mar.onnx` (0,2 MB)	Encontrar onde estão os rostos na foto	MIT
SFace (reconhecedor)	`face_recognition_sface_2021dec.onnx` (37 MB)	Transformar cada rosto em 128 números	Apache 2.0

Os dois só detectam e reconhecem: não estimam idade, gênero nem nada além do necessário (LGPD, Art. 6º, III: "limitação do tratamento ao mínimo necessário").

A configuração é exatamente a do Teste 1. A nota do SFace (89,5) só vale para a configuração medida no fotomac-lab: foto reduzida a no máximo 1920 px, confiança mínima 0,50, rosto mínimo de 10 px, alinhamento pelos 5 pontos. O adaptador do sistema foi comparado rosto a rosto com o do laboratório em 302 rostos reais, e os vetores saíram **idênticos** (cosseno 1,000000). O sistema também confere o SHA-256 dos arquivos dos modelos: um arquivo diferente geraria vetores que não se comparam com os guardados, então ele é recusado.

6.2 Do rosto ao vetor, passo a passo

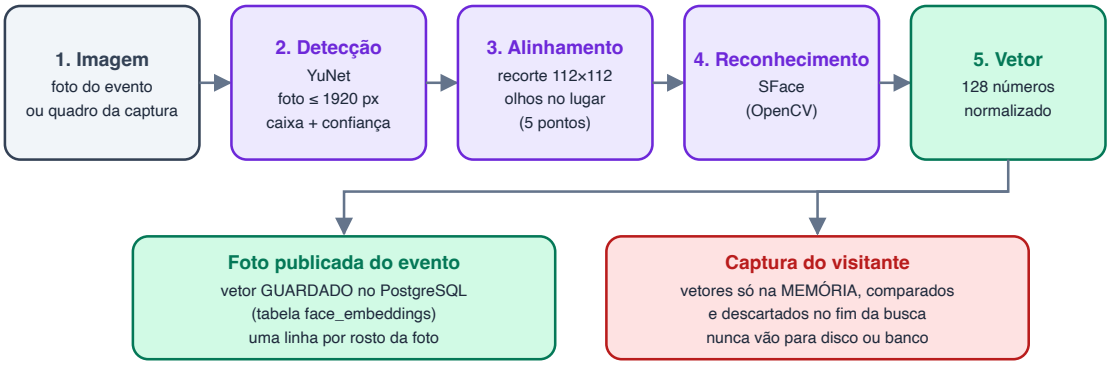


Figura 5 — O caminho de uma imagem até o vetor, e o destino diferente para a foto do evento e a captura do visitante.

1. **Foto:** o JPEG é decodificado na memória.
2. **Detecção (YuNet):** a foto é reduzida a no máximo 1920 px no lado maior, e o modelo devolve uma **caixa** em volta de cada rosto, uma **confiança** de 0 a 1 e 5 pontos do rosto. Rostos com confiança abaixo de 0,50 ou menores que 10 px são ignorados.

3. **Alinhamento:** usando 5 pontos (olhos, nariz e cantos da boca), o rosto é recortado em 112×112 pixels com os olhos sempre na mesma posição, para o reconhecedor comparar sempre "na mesma pose".
4. **Reconhecimento (SFace):** a rede neural transforma o recorte em **128 números**. Esse é o **vetor facial**, ou *embedding*: uma espécie de "impressão digital" matemática do rosto.
5. **Normalização:** o vetor é ajustado para ter comprimento 1, o que torna a comparação mais simples (próxima seção).

Uma diferença crucial: o vetor de uma **foto publicada** é guardado no PostgreSQL. Os vetores da **captura do visitante** ficam só na memória: são comparados e descartados no fim da busca, e nunca vão para disco, banco ou log. O vídeo da câmera nem sai do aparelho: o navegador envia só alguns quadros.

6.3 Como comparar dois rostos: similaridade de cosseno

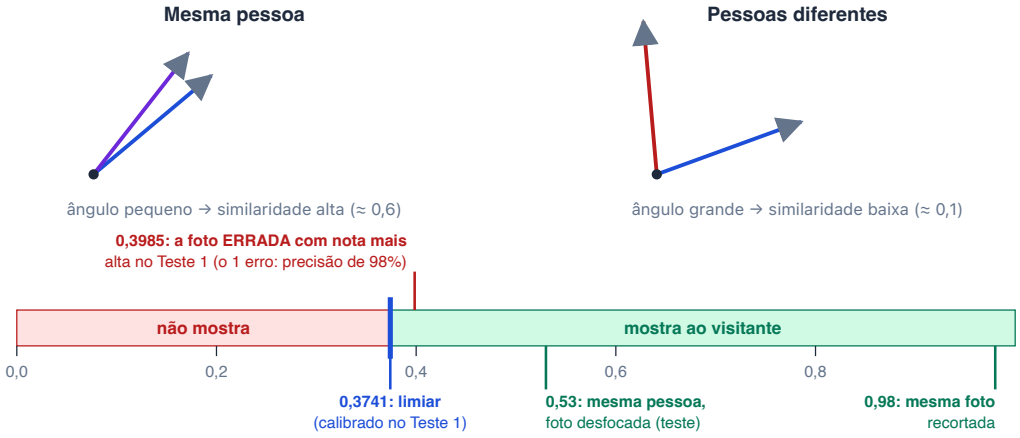


Figura 6 — A intuição das setas e a régua do limiar.

Imagine cada vetor como uma **seta** num espaço de 128 dimensões. Rostos da **mesma pessoa** geram setas que apontam **quase para o mesmo lado**; pessoas diferentes geram setas em direções diferentes. A **similaridade de cosseno** mede o quanto duas setas apontam para o mesmo lado: perto de 1, parecidíssimas; perto de 0, nada a ver.

O **limiar** é a nota de corte. Para o SFace ele vem calibrado em **0,3741**, o ponto de melhor equilíbrio do Teste 1:

- Com 0,3741, o sistema achou **48 das 59** fotos da pessoa e mostrou **1** foto errada (precisão de 98%, revocação de 81%). A foto errada de nota mais alta tirou 0,3985.
- Com 0,45, não mostrou nenhuma foto errada, mas achou só 33 das 59. Subir o limiar troca fotos encontradas por segurança.
- O sistema prefere **precisão** a **revocação**: é melhor deixar de mostrar uma foto sua do que mostrar a foto de outra pessoa para você. Mostrar a foto errada expõe um terceiro.
- Mexer no limiar muda tudo, por isso o administrador só altera depois de ler uma advertência de falso positivo, e a mudança fica registrada.

6.4 Qual rosto de cada quadro é usado

Se um quadro da captura tiver mais de uma pessoa, o sistema usa **um único rosto**, escolhido sempre do mesmo jeito (RF017):

1. Descarta rostos de baixa qualidade (confiança abaixo do mínimo, ou lado menor que 40 pixels).
2. Escolhe o rosto de **maior área**: quem está mais perto da câmera.
3. Em caso de empate: maior confiança, depois o mais perto do centro.

Assim, alguém atrás do visitante **nunca** faz aparecer as fotos de outra pessoa. A busca usa uma pose por movimento (frente, lados, cima e baixo); uma foto do evento volta se **qualquer** pose passar do limiar, o que ajuda a achar quem não estava de frente para o fotógrafo.

6.5 Como a busca fica rápida: o índice HNSW

Comparar a captura com milhares de vetores, um por um, seria lento. O pgvector usa um índice **HNSW**, uma espécie de "mapa de atalhos" entre vetores parecidos, que encontra os mais próximos sem olhar todos. A busca é sempre **filtrada por evento**: o sistema nunca compara a captura com o acervo inteiro.

6.6 Por que YuNet + SFace: o Teste 1 e a decisão do CESMAC

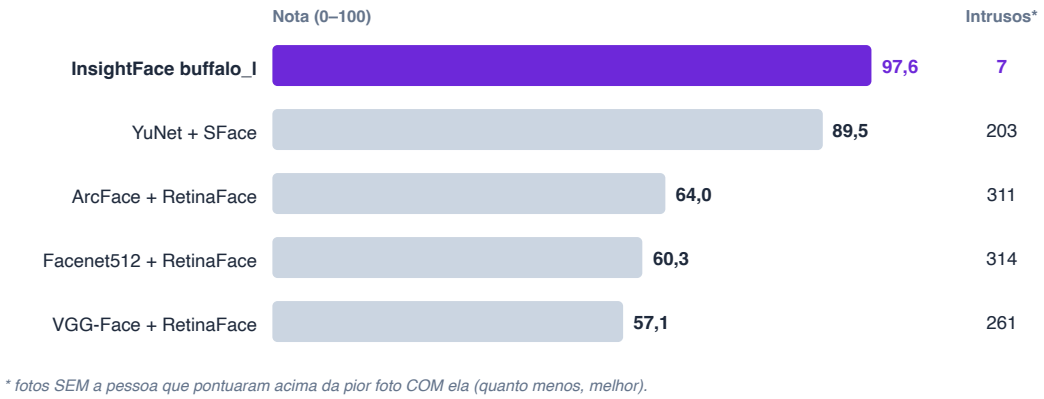


Figura 7 — Resultado do Teste 1 (fotomac-lab, 388 fotos de formatura conferidas à mão).

O que foi medido	Como
Conjunto	388 fotos reais de formatura, com plateia, perfis e rostos ao fundo
Pessoa procurada	O coordenador do curso, que aparece em dezenas de fotos
Gabarito	Cada foto conferida **à mão**: "está", "não está" ou "está, mas o rosto que casou é de outro"
Nota	100 × o melhor F1 (equilíbrio entre precisão e revocação)

O InsightFace venceu com **97,6** e só **7 intrusos** (fotos sem o coordenador que pontuaram acima da pior foto com ele). O YuNet + SFace ficou em 2º, com **89,5** e 203 intrusos.

Por que o sistema usa o 2º colocado: os modelos do InsightFace (buffalo_l) só são liberados para pesquisa não comercial; usá-los na instituição exigiria licença paga. O YuNet (MIT) e o SFace (Apache 2.0) são livres para uso institucional. Em 15/10/2026 o **CESMAC decidiu** pelo custo zero. A consequência é uma margem menor entre acerto e erro, e por isso a **calibragem com mais pessoas** virou prioridade.

O Teste 1 agora mora no sistema. A aba *Teste de precisão* (antes *Teste de qualidade*) do painel do administrador repete o teste do laboratório no acervo de verdade, só com fotos publicadas (RF029). Funciona assim:

1. Escolhe-se um evento e envia-se a foto de uma pessoa, que precisa ter autorizado (LGPD, Art. 11, I).
2. Cada modelo candidato faz a sua **rodada**: o SFace em uso, o SFace em resolução maior e o InsightFace, que entra só como régua.
3. Marca-se, uma vez só, em quais fotos a pessoa está.
4. O sistema mostra, para cada modelo, a nota (100 × melhor F1, a mesma fórmula do lab, conferida contra os resultados do Teste 1), o melhor limiar, a velocidade e o gráfico das semelhanças.

Ao encerrar o teste, a foto e as marcações são apagadas e ficam só os números.

Um achado que vale citar: o resultado **contraria as tabelas publicadas**, em que o Facenet512 aparece entre os melhores. Aquelas tabelas medem pares de rostos já recortados; a nossa tarefa é achar uma pessoa numa plateia. Conclusão: **escolher tecnologia por tabela publicada é arriscado**; é preciso medir na tarefa real.

6.7 A prova de vida (RF036)

A prova de vida responde a uma pergunta: *há uma pessoa real diante da câmera, ou uma foto?*

1. O navegador pede a **ordem dos movimentos** ao servidor, que a **sorteia**. A ordem vale para uma captura só, por 2 minutos. Assim ninguém prepara um vídeo com a ordem certa.
2. Um bonequinho mostra cada movimento; entre um e outro, o visitante **volta ao centro**.
3. O servidor mede o giro da cabeça em cada quadro pelos 5 pontos do rosto, em relação ao quadro de frente. **Decidem os giros para a esquerda e para a direita**: medidos com rosto real (17/10/2026), eles deram sinal forte (0,25 a 0,46), enquanto olhar para cima e para baixo ficou no nível do ruído (0,03 a 0,10). Cima e baixo continuam pedidos e servem como pose de busca, mas não reprovam.
4. Todos os quadros precisam ser **da mesma pessoa** (similaridade de pelo menos 0,30 com o de frente): ninguém gira a própria cabeça enquanto mostra a foto de outro.

Limite honesto (RS039): a prova de vida barra foto impressa ou na tela. Quem reunir antes fotos da mesma pessoa virando o rosto para cada lado (de redes sociais, por exemplo) pode montar a sequência, porque é o aparelho do visitante que diz qual quadro é de qual passo. Eliminar isso exige equipamento dedicado (câmera de profundidade) ou um modelo antifraude licenciado, fora do escopo. O risco está declarado nos requisitos.

---

7. Os caminhos de ponta a ponta

7.1 A vida de um evento e de uma foto

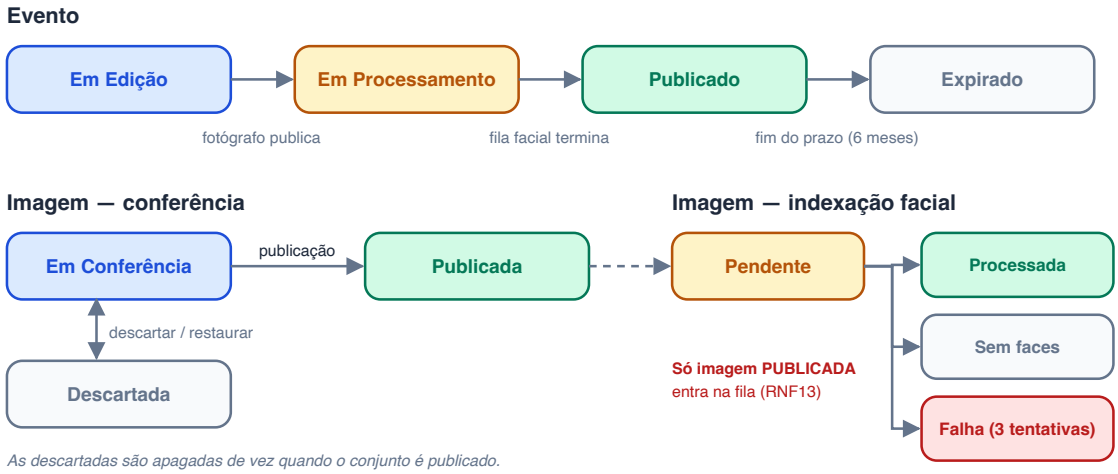


Figura 8 — Os estados do evento e da imagem.

1. O fotógrafo **cria o evento** (Em Edição) e **envia as fotos**. Em cada envio, o servidor: confere o formato pela assinatura do arquivo (JPEG, PNG, HEIC, WebP e, se o administrador ligar, RAW), recusa duplicatas, **remove o GPS**, grava o crédito do fotógrafo e o aviso de propriedade do CESMAC nos metadados e cria a miniatura e a prévia. Se a foto não é JPEG, o original fica guardado como chegou (limpo pelo ExifTool) e o sistema cria uma cópia JPEG, que é a usada nas telas, no reconhecimento facial e nos downloads.
2. Na **conferência** (aba Imagens), ele descarta o que não presta. Nada foi analisado pela IA ainda. Fotos do **fotógrafo contratado** esperam a aprovação do administrador; se o arquivo de uma foto aprovada for trocado, ela volta a esperar.
3. Ao **publicar** (todas as prontas, ou só as selecionadas), as descartadas são apagadas de vez, e as publicadas entram na **fila de indexação**. O evento fica Em Processamento.
4. O **trabalhador** (capítulo 9) gera os vetores, foto por foto. Quando a fila do evento esvazia, o evento vira **Publicado** e aparece para o público.
5. No fim do prazo de retenção (6 meses depois do evento, ajustável), o evento **expira**: fotos, cópias, capa, pacotes e vetores são eliminados. O responsável recebe um aviso por e-mail 30 dias antes.
6. O responsável pode **excluir o evento** antes disso: ele sai do site e os vetores são apagados **na hora**; as fotos esperam 7 dias, prazo em que dá para desfazer. Desfeita a exclusão, o evento fica despublicado até alguém publicar de novo.

No painel, cada evento mostra esse caminho como um cartão de **andamento** em quatro etapas (enviar → conferir e publicar → reconhecimento facial → no site), e o **sino** do cabeçalho reúne o que precisa de ação: remoções, aprovações, exclusões agendadas, falhas e prazos.

Regra que vale ouro na banca: *nenhum vetor facial existe antes da publicação.* O fotógrafo pode desistir de qualquer foto na conferência sem que o rosto de ninguém tenha virado dado biométrico.

7.2 A busca do visitante

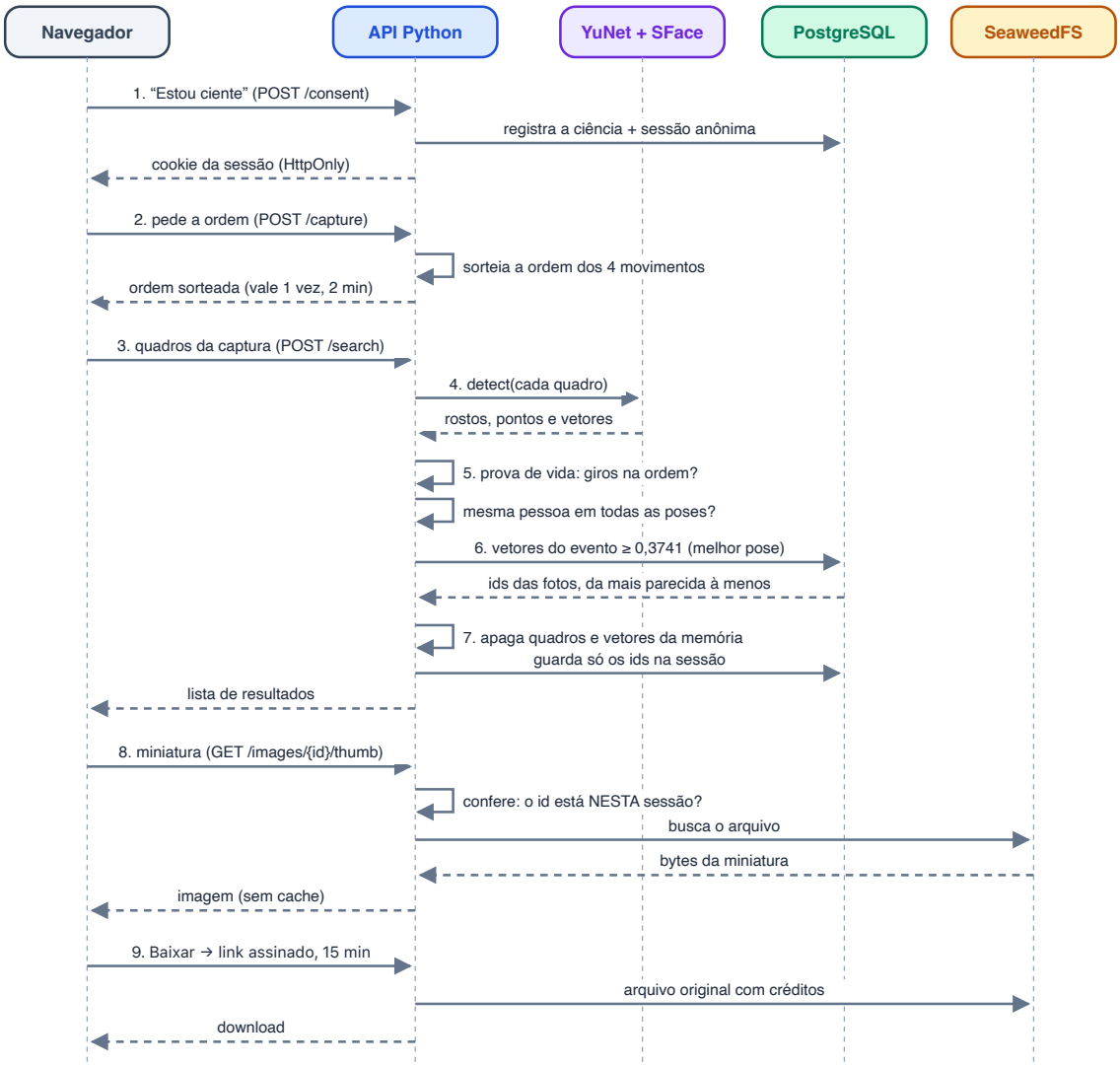


Figura 9 — O que acontece, passo a passo, quando o visitante procura as próprias fotos.

Os pontos que mostram o cuidado com a privacidade:

- **Passo 1:** sem o “Estou ciente”, **nenhum vetor é gerado**. A ciência fica registrada **sem nome e sem e-mail**. O visitante não tem conta: a sessão é anônima, num cookie.
- **Passo 2:** a ordem dos movimentos vem do **servidor**, sorteada a cada captura.
- **Passo 5:** a prova de vida (capítulo 6.7) é conferida **no servidor**; reprovada, não há busca.
- **Passo 7:** os quadros e os vetores são **apagados da memória** ao fim da função. A sessão guarda **só a lista de ids** das fotos encontradas.
- **Passo 8:** cada miniatura só é entregue se o id **está no resultado desta sessão**. Quem tentar adivinhar o endereço de outra foto recebe “não encontrada”.

- **Passo 9:** o download é feito por um **link assinado** (HMAC), válido por 15 minutos e preso à sessão: copiado para outra pessoa, ele não funciona. O arquivo sai pela API, e o endereço do SeaweedFS nunca chega ao navegador. A foto baixada é uma **cópia com a marca d'água "CESMAC"** no canto escolhido pelo administrador; o original do acervo não muda, e as contas da equipe baixam sem marca.
- **Ao sair da página**, o resultado é apagado. Pense num computador da biblioteca: a próxima pessoa não pode ver as suas fotos.
- **Limites:** 10 buscas por sessão e até 200 fotos por busca, além de limites por endereço, contra quem tentar varrer o acervo com um robô.

7.3 Remoção a pedido do titular

O visitante pode pedir a remoção de uma foto **pela própria foto encontrada**, ampliada na tela. Quem não usou o sistema procura o encarregado pelo tratamento de dados. A foto **sai da busca na hora**, o fotógrafo e os administradores recebem e-mail, e, se ninguém agir, o **sistema exclui sozinho em 72 horas**. Ao concluir, o e-mail de contato do visitante também é apagado.

7.4 Pessoa institucional (RF020 e RF021)

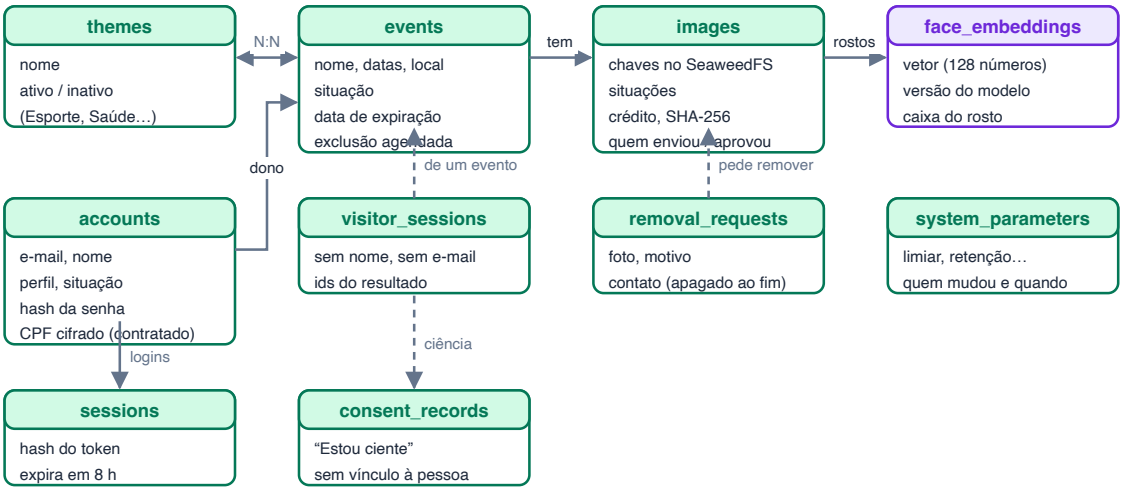
Coordenadores, diretores e reitoria podem autorizar o sistema a achá-los em **todos** os eventos, para a comunicação do CESMAC. É a única busca entre eventos que o FOTOMAC faz, e só existe porque a própria pessoa assinou.

1. **O administrador cadastra** nome, cargo, e-mail institucional e de 1 a 5 fotos do rosto. As fotos são guardadas **sem metadados** e **não são processadas**: o próprio banco recusa vetor de quem ainda não autorizou (gatilho na tabela `institutional_vectors`).
2. **A pessoa recebe um link de uso único** (7 dias), lê o termo, vê as fotos que serão usadas e **assina desenhando na tela**. Também pode recusar, e aí tudo é apagado. Sem assinatura em 30 dias, o cadastro some sozinho.
3. **Assinado**, o sistema guarda o texto exato do termo, a data e a hora, o IP, o navegador e um **código de verificação** (SHA-256) que muda se qualquer palavra, foto ou traço for alterado. O administrador baixa o **formulário oficial em PDF**.
4. **O trabalhador procura a pessoa no acervo publicado inteiro**, sem o teto de 200 da busca do visitante, e daí em diante compara cada foto nova assim que ela é indexada.
5. **Fotógrafo, marketing e administrador consultam pelo nome**: veem os eventos com "N de M imagens" e **confirmam ou rejeitam** cada marcação, porque semelhança não é identidade. Toda consulta fica registrada.
6. **A pessoa revoga** quando quiser, pelo link do e-mail de confirmação: fotos, vetores e marcações são eliminados na hora. O termo assinado fica como prova.

A assinatura é **eletrônica** (desenhada na tela, com registro), e não "digital" com certificado ICP-Brasil. Se ela basta é decisão do jurídico do CESMAC (pendência 5 dos requisitos).

---

8. O banco de dados



Outras tabelas de apoio: password_tokens (links de ativação e recuperação), package_jobs (ZIPs), reindex_jobs e reindex_items, recognition_tests (teste de p

Figura 10 — As principais tabelas e como se relacionam (simplificado).

As tabelas do fichário:

Grupo	Tabelas	Observação
Equipe	`accounts`, `sessions`, `password_tokens`	A senha é guardada só como **hash Argon2id**; da sessão, só o hash do token. CPF e nascimento do contratado, **cifrados** pelo pgcrypto, com a chave fora do banco
Acervo	`events`, `themes`, `images`	A foto em si está no SeaweedFS; aqui ficam as chaves, as situações, a exclusão agendada e quem enviou e aprovou cada foto
Biometria	`face_embeddings`	**Tabela separada** para o dado sensível, com a versão do modelo em cada linha
Visitante	`consent_records`, `visitor_sessions`, `removal_requests`	Sem nome, sem conta; o contato da remoção é apagado ao concluir
Operação	`system_parameters`, `package_jobs`, `reindex_jobs`, `reindex_items`	Parâmetros, ZIPs e reindexações
Calibração	`recognition_tests` e tabelas filhas	O teste de precisão; ao encerrar, ficam só os números
Prova	`audit_log`	**Inalterável**: um gatilho do banco recusa alterar ou apagar

Por que o `audit_log` é inalterável no próprio banco, e não só no código Python? Se a regra estivesse só no Python, qualquer pessoa com acesso ao banco poderia apagar um registro incômodo. Com o gatilho, nem o sistema consegue. É isso que torna o registro uma **prova** (LGPD, Art. 6º, X: responsabilização e prestação de contas).

---

9. O trabalhador em segundo plano

Gerar vetores leva tempo: cada foto custa uma fração de segundo, e um evento pode ter até 1.000 fotos. Se a API fizesse isso enquanto o fotógrafo espera, a tela travaria. Por isso existe o **trabalhador**: um laço que roda **dentro da API**, numa linha de execução separada (*thread*), e processa as tarefas na ordem.

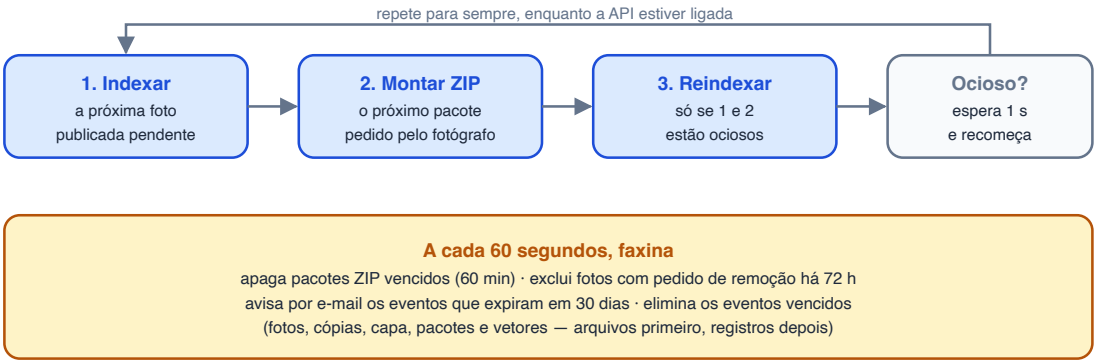


Figura 11 — O laço do trabalhador e a faxina periódica.

- **A fila é o próprio banco.** Uma foto "pendente" é uma linha com `indexing_status = 'pending'`. Não há programa de fila extra para instalar.
- **`FOR UPDATE SKIP LOCKED`** é o truque que permite ligar vários trabalhadores ao mesmo tempo: cada um "tranca" a foto que pegou, e os outros pulam as trancadas. O mesmo código que roda no seu notebook escala para vários servidores.
- **Uma foto com defeito nunca para a fila:** são até 3 tentativas, com intervalo crescente, e depois a foto fica marcada como "Falha".
- **Prioridade:** a reindexação só anda quando não há publicação esperando. O fotógrafo nunca espera por uma tarefa de manutenção.
- **A faxina de cada minuto:** pacotes vencidos, remoções de 72 h, testes de precisão esquecidos, avisos e eliminação por expiração, exclusões cujo prazo acabou, contas temporárias que venceram e CPF de contratado que já passou do prazo de guarda.
- **Eliminação consistente:** ao apagar uma foto, os **arquivos são apagados primeiro** e a linha do banco **depois**. Na ordem inversa, uma falha no meio deixaria uma foto "órfã" no almoxarifado, sem ficha, que ninguém mais acharia para apagar.

---

10. Privacidade desde o desenho (LGPD)

A LGPD não foi uma revisão no final: ela decidiu a arquitetura. As regras que viraram código:

Princípio / artigo (lido na lei)	Como virou código
Dado sensível (Art. 5º, II: "dado … biométrico, quando vinculado a uma pessoa natural")	Vetor em tabela própria; nunca em log; captura do visitante nunca armazenada
Consentimento específico e destacado (Art. 11, I)	"Estou ciente" **antes** da câmera, sem caixa pré-marcada, conferido no servidor
Finalidade (Art. 6º, I)	A busca serve a um evento; não existe busca no acervo inteiro
Necessidade (Art. 6º, III)	Idade e gênero desligados (o teste de idade foi medido, descartado e retirado do sistema); GPS removido; visitante sem conta; do contratado, só CPF e nascimento
Término e eliminação (Art. 15 e 16)	Expiração automática com eliminação de fotos, pacotes e vetores; ao excluir um evento, os vetores saem na hora
Segurança (Art. 46)	Permissão conferida no servidor; sessão em cookie HttpOnly; senha em Argon2id; CPF cifrado; escrita só da própria origem; SeaweedFS privado
Prestação de contas (Art. 6º, X)	Registro de auditoria inalterável

Limite honesto: isto é engenharia com a lei na mão, não parecer jurídico. A base legal final, a Política de Privacidade e a indicação do encarregado (DPO) são decisões da área jurídica do CESMAC, e isso deve ser dito na banca.

---

11. Como provamos que funciona

Tipo de teste	Quantos	O que prova	Comando
Testes do servidor (pytest)	215	Cada regra de negócio e de privacidade	`cd backend && .venv/bin/pytest`
Testes de ponta a ponta (Playwright)	8 roteiros	Um navegador de verdade clica nas telas, com o YuNet + SFace real	`node e2e/visitor.e2e.mjs …`
Verificação de tipos (TypeScript)	—	O site não tem erro de tipo	`npm run typecheck`

Alguns testes que valem citar, porque provam a **regra** e não só o funcionamento:

- **Sem ciência, sem busca:** o servidor recusa a busca sem o "Estou ciente", mesmo que alguém pule a tela.
- **Foto não publicada não entra na fila.**
- **Outra sessão não abre a foto:** o link de download de uma sessão dá erro em outra.
- **O limiar funciona de verdade:** uma foto desfocada da mesma pessoa (similaridade 0,53) aparece com limiar 0,3741 e **some** com 0,60.
- **Troca de modelo segura:** um modelo que não carrega nunca vira o modelo em uso; depois de uma troca, a foto não aparece até ser reindexada, e os vetores antigos somem.
- **Prova de vida:** uma captura sem os giros na ordem sorteada é recusada, e a ordem não vale duas vezes.
- **Aprovação não se herda:** trocar o arquivo de uma foto aprovada do contratado a devolve para aprovação.
- **Origem estranha é recusada:** uma ação vinda de outro endereço do CESMAC recebe 403.
- **Os testes não usam fotos do CESMAC:** usam retratos da NASA em domínio público.

Teste de mutação: em alguns pontos, eu **estraguei o código de propósito** para ver se o teste reclamava. Se o teste continua passando com o código errado, ele não protege nada. É a forma de provar que o teste é bom.

Os testes do servidor nunca tocam os seus dados: usam um banco separado, o `fotomac_test`. Os de ponta a ponta usam o banco de desenvolvimento, e por isso o script `cleanup_e2e.py` apaga tudo o que eles criaram.

---

12. Onde está cada coisa no código

```
FOTOMAC (prototype)/
├── backend/                    a API em Python
│   ├── app/
│   │   ├── main.py             liga tudo: rotas, banco, modelo
│   │   ├── config.py           lê o .env (endereços, senhas)
│   │   ├── api/                as rotas (endereços da API)
│   │   ├── face/               PORTA + ADAPTADOR do rosto
│   │   │   ├── port.py         a "tomada" (contrato)
│   │   │   ├── sface_engine.py o "plugue" (YuNet + SFace)
│   │   │   ├── factory.py      o builder: catálogo e modelo em uso
│   │   │   ├── switch.py       a troca de modelo pelo painel
│   │   │   └── selection.py    qual rosto de cada quadro usar
│   │   ├── storage/            PORTA + ADAPTADOR dos arquivos
│   │   ├── indexing/           o trabalhador e a reindexação
│   │   ├── calibration/        teste de precisão e prova de vida (poses)
│   │   ├── public/             regras do visitante
│   │   ├── events/             eventos, exclusão com prazo, avisos do sino
│   │   ├── images/ accounts/ …  regras do painel
│   │   ├── security/           links assinados (HMAC)
│   │   ├── parameters.py       parâmetros (RF026)
│   │   └── db/models.py        as tabelas
```

```
│   ├── migrations/              0001 … 0024: história do banco
│   ├── tests/                   os 215 testes
│   └── scripts/                 1º administrador, limpeza
├── frontend/                    o site em Expo
│   ├── src/app/                 as telas (arquivo = endereço)
│   │   ├── index.tsx            início público do visitante
│   │   ├── evento/[id].tsx      página do evento + busca
│   │   ├── login.tsx            entrada da equipe
│   │   └── (painel)/            fotógrafo, marketing, contratado e administrador
│   ├── src/api/client.ts        a única porta para a API
│   ├── src/components/          peças de tela
│   └── e2e/                     os testes de navegador
├── infra/                       liga PostgreSQL e SeaweedFS
└── docs/                        relatórios (este guia etc.)
```

Três arquivos para ler com calma antes da banca:

1. `backend/app/face/port.py`, para explicar o adaptador.
2. `backend/app/public/service.py`, função `search`, para explicar a busca com todas as proteções, e `backend/app/calibration/poses.py`, para a prova de vida.
3. `backend/app/indexing/worker.py`, para explicar a fila.

---

13. Limitações que você deve conhecer

Reconhecer os limites na banca passa confiança. Os do sistema hoje:

1. **O Teste 1 mediu uma pessoa só.** A ordem do ranking é confiável; os números exatos, menos. A calibragem com mais pessoas (RNF18) é trabalho futuro.
2. **Margem do SFace:** no Teste 1, 203 fotos sem a pessoa pontuaram acima da pior foto com ela (no InsightFace, 7). Com o limiar 0,3741, a cada ~49 fotos mostradas, 1 pode ser de outra pessoa. O administrador pode subir o limiar, e a calibragem com mais pessoas é a prioridade.
3. **E-mails de desenvolvimento:** nenhum e-mail é enviado de verdade; cada mensagem vira um arquivo na pasta `backend/outbox/`. Em produção, entra um adaptador de e-mail real e sai o link de demonstração do painel (decidido para o fim do projeto).
4. **Tudo roda numa máquina só, sem HTTPS.** No servidor do CESMAC, o cookie passa a exigir HTTPS (`cookie_secure=True`) e os endereços mudam só no `.env`.
5. **Os limites de requisição ficam na memória da API:** ao reiniciar a API, a contagem zera. Com vários servidores, seria preciso guardá-los num lugar comum.
6. **O trabalhador roda dentro da API.** Basta para o protótipo; em produção, ele pode rodar como um processo separado com `python -m app.indexing.worker`.
7. **A prova de vida tem um limite** (capítulo 6.7): barra foto impressa ou na tela, mas não um conjunto de fotos da pessoa em poses diferentes, preparado antes (RS039).
8. **Crianças e adolescentes, sem regra por enquanto:** em 19/10 tudo o que tratava de menores saiu do sistema (evento infantil, borrão, avisos de idade), até a resposta oficial do orientador e da coordenação. Até lá, não se enviam fotos de eventos com crianças.

14. Perguntas que a banca pode fazer

"A busca da pessoa institucional não fere a regra de buscar só dentro do evento?" É a única exceção, e ela tem dono: só vale para quem assinou o termo, que diz exatamente isso. Não existe busca por foto de uma pessoa qualquer: a consulta é só pelo nome de quem já autorizou, e cada consulta fica registrada. Quem revoga some de todas as listas na hora.

"Por que não guardar a captura do visitante?" Porque não há finalidade para guardá-la. Ela só serve para gerar os vetores e comparar. Guardar seria tratar dado biométrico além do necessário (Art. 6º, III), e um rosto vazado não se troca como uma senha.

"E se alguém mostrar a foto de outra pessoa para a câmera?" A prova de vida pede giros de cabeça numa ordem sorteada pelo servidor e exige a mesma pessoa em todos os quadros: uma foto parada não gira. O limite honesto é o conjunto de fotos da pessoa em várias poses, preparado antes; esse risco está declarado (RS039) e eliminá-lo exige equipamento dedicado.

"E se surgir um modelo melhor que o YuNet + SFace?" Escreve-se um adaptador novo e acrescenta-se uma linha no catálogo do builder. O administrador troca o modelo no painel: o sistema carrega o modelo novo antes, volta o limiar ao calibrado dele e reindexa o acervo. Até ser reindexada, uma foto não aparece na busca, porque vetores de modelos diferentes não se comparam. Foi exatamente o que aconteceu na troca do InsightFace pelo SFace.

"Por que não ficaram com o InsightFace, se ele venceu o teste?" Licença. Os modelos dele só são liberados para pesquisa não comercial; o YuNet e o SFace são livres para uso institucional. A decisão foi do CESMAC, pelo custo zero. O preço é uma margem menor, compensada com calibragem e com o limiar ajustável pelo administrador.

"Como você garante que um visitante não vê a foto de outro?" Três camadas: a busca é por um único evento; a sessão só guarda os ids encontrados para aquele rosto; e toda imagem é entregue pela API, que confere se o id está naquela sessão. O endereço do armazenamento nunca chega ao navegador.

"Qual é a precisão do sistema?" No Teste 1, com 388 fotos conferidas à mão, o YuNet + SFace teve nota 89,5 (AUC de 0,970). No limiar 0,3741 ele achou 48 das 59 fotos da pessoa e mostrou 1 foto errada (precisão de 98%). A ressalva honesta: foi uma pessoa só, e a calibragem com mais pessoas é a próxima etapa.

"Por que um limiar de 0,3741?" É o ponto de melhor equilíbrio (F1) do Teste 1 para o SFace: 48 de 59 fotos achadas, 1 errada. Com 0,45 não houve erro, mas só 33 fotos foram achadas. O administrador pode subir o valor depois de ler uma advertência, e a mudança fica registrada.

"Por que Python no servidor e não JavaScript?" As bibliotecas de reconhecimento facial e a bancada de testes (OpenCV, fotomac-lab) são nativas de Python. Colocar a IA no mesmo processo da API evita uma comunicação extra entre programas.

“Por que web antes do mobile?” A API já é o contrato único (`/api/v1`): o aplicativo móvel vai consumir as mesmas rotas, e o Expo permite reaproveitar boa parte das telas. Começar pela web entrega primeiro o fluxo completo.

“E se o servidor cair no meio da indexação?” A fila está no banco. Ao religar, o trabalhador continua de onde parou: a foto que estava sendo processada continua como “pendente”, porque a transação não foi confirmada.

“O sistema reconhece menores de idade?” O sistema **não estima idade**. Um modelo de idade chegou a ser medido como ajuda ao fotógrafo e foi **descartado** (deu cerca de 54 anos a uma criança). Em 19/10, todas as regras sobre menores (evento infantil, borrão e avisos de idade) foram retiradas: a forma certa de tratar crianças depende da LGPD (Art. 14) e foi levada ao orientador e à coordenação, com as alternativas estudadas. Até a resposta oficial, fotos de eventos com crianças não entram no sistema.

“Por que o administrador aprova as fotos do fotógrafo contratado?” Porque o contratado não é da equipe e trabalha só um dia. A aprovação barra conteúdo impróprio, e cada foto guarda quem a enviou e quem a aprovou. Se o arquivo for trocado depois, a aprovação cai, porque ela vale para o conteúdo que o administrador viu.

---

15. Glossário

Sistema e infraestrutura

Termo	Em uma frase
API	O programa que recebe pedidos e devolve respostas; aqui, o “gerente” do sistema
HTTP / JSON	O idioma (HTTP) e o formato das mensagens (JSON) entre o site e a API
Endpoint / rota	Um endereço da API, como `POST /api/v1/events/{id}/search`
Cookie HttpOnly	Um crachá guardado pelo navegador que o JavaScript não consegue ler
ORM (SQLAlchemy)	Ferramenta que transforma tabelas do banco em classes Python
Migração	Um passo numerado na evolução da estrutura do banco
S3 / bucket / objeto	O protocolo de armazenamento de arquivos; o “balde”; o arquivo
Porta e adaptador	A tomada (contrato) e o plugue (implementação), para trocar peças sem quebrar o resto

Reconhecimento facial, segurança e testes

Termo	Em uma frase
Detecção	Achar onde estão os rostos numa imagem
Reconhecimento	Transformar um rosto num vetor que permite comparar pessoas
Vetor / embedding	A lista de 128 números (no SFace) que representa um rosto
Similaridade de cosseno	O quanto dois vetores apontam para o mesmo lado (de 0 a 1, na prática)
Limiar	A nota de corte da similaridade para mostrar uma foto
Precisão / revocação / F1	Acertei o que mostrei? / Achei tudo o que existia? / O equilíbrio das duas
Falso positivo	Mostrar uma foto em que a pessoa **não** está, o erro mais grave aqui
HNSW	O índice de "atalhos" que acelera a busca de vetores parecidos
Fila / trabalhador	A lista de tarefas pendentes e o laço que as executa em segundo plano
Transação	Um grupo de alterações no banco que vale inteiro ou não vale nada
HMAC / link assinado	Uma assinatura que só o servidor sabe gerar e que prova que o link é legítimo
Hash (Argon2id)	Uma transformação sem volta; guarda a senha sem que ninguém possa lê-la
Teste de ponta a ponta (e2e)	Um robô usa o site num navegador real, como uma pessoa usaria
Teste de mutação	Estragar o código de propósito para provar que o teste percebe
Prova de vida	Conferir que há uma pessoa real diante da câmera, e não uma foto
CSRF	Um site fazer o navegador de quem está logado enviar uma ação sem a pessoa saber